VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

OBJECT ORIENTED PROGRAMMING
(25MC409)
I M.C.A II Semester
2025-26 EVEN SEMESTER

Name of the Faculty : Dr. K. Gayatri
Assistant Professor, Department of CA
VFSTR (Deemed to be University)

Institute Vision & Mission

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

VISION:

To evolve in to a centre of excellence in Science & Technology through creative and innovative practices in teaching-learning, towards promoting academic achievement and research excellence to produce internationally accepted, competitive and world class professionals who are psychologically strong & emotionally balanced imbued with social consciousness & ethical values.

MISSION:

To Provide High Quality academic programmes, training activities, research facilities and opportunities supported by continuous industry - institute interaction aimed at promoting employability, entrepreneurship, leadership and research aptitude among students and contribute to the economic and technological development of the region, state and nation.

Department Vision & Mission

VISION:

To emerge as a renowned center in the field of computer applications for providing high quality education and research to produce globally competent professionals with ethical values.

MISSION:

- **M1:** To provide high quality education with cutting-edge curriculum and learner-centered approach.
- **M2:** To establish the culture of innovation and research through industry-academia linkages.
- **M3:** To cultivate ethical and socially-conscious professionals with leadership qualities, who understand the global implications of technology.

PO' s of the Department

- **PO1 (Foundation Knowledge):** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.
- **PO2 (Problem Analysis):** Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.
- **PO3 (Development of Solutions):** Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- **PO4 (Modern Tool Usage**): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO' s of the Department

- **PO5 (Individual and Teamwork):** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- **PO6 (Project Management and Finance)**: Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.
- **PO7 (Ethics):** Commit to professional ethics in managing software projects with financial | aspects. Learn to use new technologies for cyber security and insulate customers from malware
- **PO8 (Life-long learning):** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

PEO' s of the Department

- **PEO1:** Develop cutting-edge technical skills to succeed in career and as an entrepreneur.
- **PEO2 :** Pursue higher education and research with critical thinking and problem solving capabilities
- **PEO3:** Demonstrate ethical, collaborative, and leadership abilities in multi-disciplinary domains.

PSO's of the Department

- PSO1: To provide solutions for complex computational problems with requisite mathematical, programming languages and tools.
- PSO2: To design, develop and analyse software and related services using algorithms, web design, and Big Data Analytics for real time applications.

Module-2

Unit-2

MULTITHREADING: Thread, Thread life cycle, Thread creation, Thread priorities, multithreading, Thread Synchronization, Deamon Thread.

Threads

- A thread is a lightweight sub process, a smallest unit of processing. It is a separate path of execution.
- Threads are independent, if there occurs exception in one thread, it doesn't affect other threads. It shares a common memory area.
- Every threads in Java is created and controlled by the java.lang thread class.

Program

10,000
Lines of code

Line by Line
Execution

10 hours

Multi-tasking

Thread based
Multi-tasking

Program

5000 lines

Wait to complete

5000 lines

Multi-tasking

Thread based Multi-tasking

Independent of each other

Why to wait for first part (5000 lines) till it get executed?

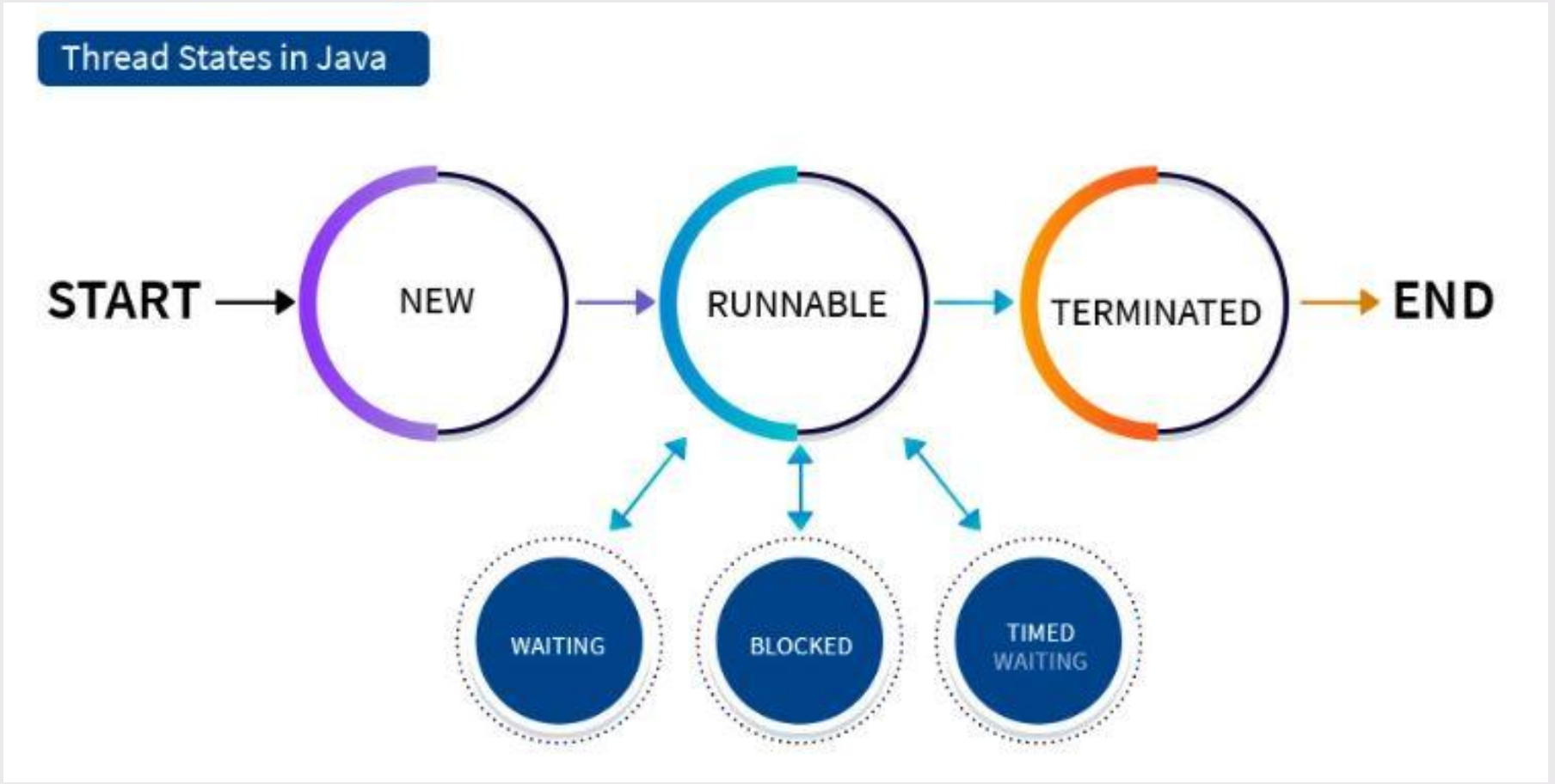
We can execute two parts simultaneously

Thread Life Cycle

Thread States (or) Lift-Cycle of a Thread

- The life cycle of a thread contains several states. At any time, the thread falls into any one of the states.
- At any time, a thread is said to be in one or more states.
  - New/Born
  - Runnable
  - Running
  - Blocked
  - Dead

The life cycle of the thread in java is controlled by JVM



New Thread:

When a new thread is created, it is in the new state. The thread has not yet started to run when thread is in this state.

When a thread lies in the new state, its code is yet to be run and hasn't started to execute.

Runnable State:

A thread goes through this state after the invocation of the start() method. At this state, a thread is considered alive because it is either running or ready for execution, but waiting for resource allocation.

In a multi-threaded environment, Thread Scheduler allocates a time slot for each thread. So, each thread runs for a particular amount of time and then hand over the resource to other threads in the runnable state.

Non-Runnable (Blocked)

- This is the state when the thread is still alive, but is currently not eligible to run.

A thread becomes "Not Runnable" when one of these events occurs:

- If sleep method is invoked.
- The thread calls the wait method.
- The thread is blocking on I/O.

Terminated

- A thread is in terminated or dead state when its run () method exits.

Methods

- **String getName( ):** It obtains the name of the thread.
- **void setName(String str):** It sets the name of the thread.
- **int getPriority( ):** It obtains the priority of the thread.
- **void setPriority(int):** It sets the priority of the thread.
- **boolean isAlive( ):** Determines if a thread is still running or not. It returns true if the thread upon which it is called is still running. Otherwise, it returns false.
- **void sleep(long milliseconds):** Suspend a thread for a period of time.
- **void start( ):** Start a thread by calling its run method.

- **void run( ):** It defines the code that constitutes the new thread. When ever the start( ) method executes it automatically call of run( ) method.
- **void join( ):** This method waits until the thread on which it is called terminates.
- **Thread currentThread( ):** It returns a reference to the currently executing thread.
- **void yield( ):** A thread can call the yield method to give other threads a chance to execute.
- **ThreadGroup getThreadGroup( ):** It returns the name of the thread group which contain currently running thread. ThreadGroup represents set of Threads. It has one constructor method as ThreadGroup(String name).

Main Thread:

- When a java program starts up, one thread begins running immediately. This one is **main thread**, which is created by virtual machine. It is executed in the program for the first time. It must be the last thread to finish execution. When the main thread stops the program terminates.
- Even though the main thread is automatically created when the program starts, it can be controlled through Thread object. For this we obtain a reference to by calling the method "**currentThread( )**".

```
class demo
{
    public static void main(String[] args)
    {
    Thread
obj=Thread.currentThread();
    System.out.println("current="+obj);
    obj.setName("india");
    System.out.println("after="+obj);
    }
}
```

O/P:

```
current=Thread[main,5,main]
after=Thread[india,5,main]
```

```
class demo
{
public static void main(String[] args)
    {
    Thread obj=Thread.currentThread();
    try
    {
    for(int n=5;n>0;n--)
    {
    System.out.println(n);
    obj.sleep(1000);
    }
    }
    catch(InterruptedException e)
    {
    System.out.println("exception");
    }
    }
 }
```

Creating threads

Threads can be created by using two mechanisms:

1. **Implementing the Runnable Interface**

2. **Extending the Thread class**

Implements the Runnable interface: Runnable is a system defined interface. Once we create a class, it implements the Runnable interface forms a thread class. To implement Runnable, a class need only implement a single method called **run( )**.

Implementing the Runnable Interface


```
class MyRunnable implements Runnable
{
    public void run()
    {
        for(int i =0;i<100;i++)
        {
            System.out.println("Child Thread");
        }
    }
}
```

Defining a thread

Job of a thread

```
public class Vignan
{
    public static void main(String[] args)
    {
        MyRunnable r = new MyRunnable();

     Thread t = new Thread(r);

     t.start();
     }
}
```

Thread instantiation

Target run method

Starting of a thread

Another way: `Thread t = new Thread(new MyRunnable());`

Extending the Thread class: The second way to create a thread is to create a new class that extends **Thread**, and then create an instance of that class. The extending class must override the run( ) method. It must also call start( ) to begin execution of the new thread.

Extending the Thread class


```
class MyThread extends Thread
{
    public void run()
    {
        for(int i =0;i<100;i++)
        {
            System.out.println("Child Thread");
        }
    }
}
```

Defining a thread

Job of a thread

```
public class Vignan
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();

        t.start();

        for(int i =0;i<1000;i++)
        {
            System.out.println("Main Thread");

        }
    }
}
```

Thread instantiation

Starting of a thread

Thread Priorities

- ✓ Every thread in Java has some priority. It may be default priority generated by JVM or Customized priority set by programmer

- ✓ Priority of a thread describes how early it gets execution and selected by the thread scheduler.

- ✓ The valid range of thread priorities is 1 to 10

Where 1 is min priority and 10 is max priority

- ✓ Thread class defines the following constants to represent some standard priorities
  - ❑ Thread.MIN_PRIORITY (value is 1)
  - ❑ Thread.NORM_PRIORITY (value is 5)
  - ❑ Thread.MAX_PRIORITY (value is 10)

Get and Set methods in Thread priority

- ❑ getPriority () method is used to get the priority of a thread.
- ❑ setPriority () method is used to set the priority of a thread.

```
class Demo extends Thread
{
    public void run()    {
        for(int i=1;i<=3;i++)
        System.out.println(getName()+" "+i);   }
}
class ThprDemo
{   public static void main(String[] args)
    {
        Demo obj=new Demo();
        Demo obj1=new Demo();
        Demo obj2=new Demo();
        obj.setPriority(Thread.MAX_PRIORITY);
        obj1.setPriority(Thread.MIN_PRIORITY);
        obj2.setPriority(Thread.NORM_PRIORITY);
        obj.start();
        obj1.start();
        obj2.start();
    }
}
```

O/P:

```
Thread-1 1
Thread-1 2
Thread-1 3
Thread-3 1
Thread-3 2
Thread-3 3
Thread-2 1
Thread-2 2
Thread-2 3
```

yield(), sleep() and join() Methods

A yield() method can stop the currently executing thread and will give a chance to other waiting threads of the same priority.

A join() method in Java allows one thread to wait until another thread completes its execution. In simpler words, it means it waits for the other thread to die.

A sleep() method can be used to pause the execution of the current thread for a specified time in milliseconds.

Multi threading

Multi threading

- Java provides built-in support for *multithreaded programming*.
- A multithreaded program contains two or more parts that can run concurrently. Each part is called a Thread.

Advantages

- ✓ It doesn't block the user because threads are independent and you can perform multiple operations at same time.
- ✓ You can perform many operations together so it saves time.
- ✓ Threads are independent so it doesn't affect other threads if exception occur in a single thread.

Creating Multiple threads

```
class A extends Thread
{
        public void run()
        {
                try
                {
                    for(int i=1;i<=3;i++)
                    {
                        System.out.println("A - "+i);
                        Thread.sleep(1000);  // thread to sleep for 1 second
                    }
                }
                catch(InterruptedException ie)
                {
                }
         }
}
```

```
class  B extends Thread
{
        public void run()
        {
              try
              {
                  for(int i=1;i<=3;i++)
                      {
                          System.out.println("B - "+i);
                          Thread.sleep(2000);        // thread to sleep for 2 seconds
                      }
                }
              catch(InterruptedException ie)
              {
              }
         }
}
```

```
class  Thread1
{
    public static void main(String args[])
    {
        A t1=new A();
        B t2=new B();
        t1.start();
        t2.start();
    }
}
```

```
class Message extends Thread
{
    String msg;
    int time;
    Message(String msg, int time)
    {
        this.msg = msg;
        this.time = time;
    }
    public void run()
    {
        while(true)
        try
        {
            Thread.sleep(time);
            System.out.println(msg);
        }
        catch (Exception err){}
    }
}
```

```
class Threademo
{
    public static void main(String[] args)
    {
        System.out.println("Press Ctrl+c to exit.\n");
        Message T1 = new Message("Good Morning", 1000); // 1000 milliseconds
        Message T2 = new Message("Hello", 2000);
        Message T3 = new Message("Welcome", 3000);
        T1.start();
        T2.start();
        T3.start();
    }
}
```

Multitasking

- ✓ All modern operating systems support the concept of multitasking.
- ✓ Multitasking is the feature that allows the computer to run two or more programs concurrently.
- ✓ CPU scheduling deals with the problem of deciding for which of the process, the ready queue is to be allocated.
- ✓ It follows CPU (Processor) scheduling algorithms.

Process-based Multitasking (Multiprocessing)

- Each process have its own address in memory i.e. each process allocates separate memory area.
- Process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another require some time for saving and loading registers, memory maps, updating lists etc.

Thread-based Multitasking (Multithreading)

- ✓ Threads share the same address space
- ✓ Thread is lightweight.
- ✓ Cost of communication between the thread is low.

Difference between Multiprocessing and Multithreading

Multithreading	Multiprocessing
More than one thread running simultaneously	More than one process running simultaneously
Its part of a program	Its part of a program
It's a light weight process	It's a heavy weight process
Threads are divided into sub threads	Process is divided into threads
Within the process threads are communicated	There is no communications between processors directly

Note:

1. Java provides built-in support for multithreaded programming
2. Java multithreading is a platform independent
3. Elimination of main loop/polling is the main benefit of multithreaded programming.

Priority Scheduling:

- The priority scheduling follows on Sun Solaris systems. But it doesn't work on the present operating systems like windows NT etc.,
- The thread with a piece of system code is called the thread scheduler. It determines which thread is running actually in the CPU.
- The Solaris java platform runs a thread of given priority to completion or until a higher priority thread becomes ready at that point preemption occurs.

Process	Arrival Time	Burst Time	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

Round-Robin Scheduling:

- Round Robin scheduling is available on Windows 95 and Windows NT is time sliced.
- Java supports thread implementation depending on round robin scheduling. Here, each thread is given a limited amount of time called time quantum.

Process	Burst time
P1	24
P2	10
P3	03

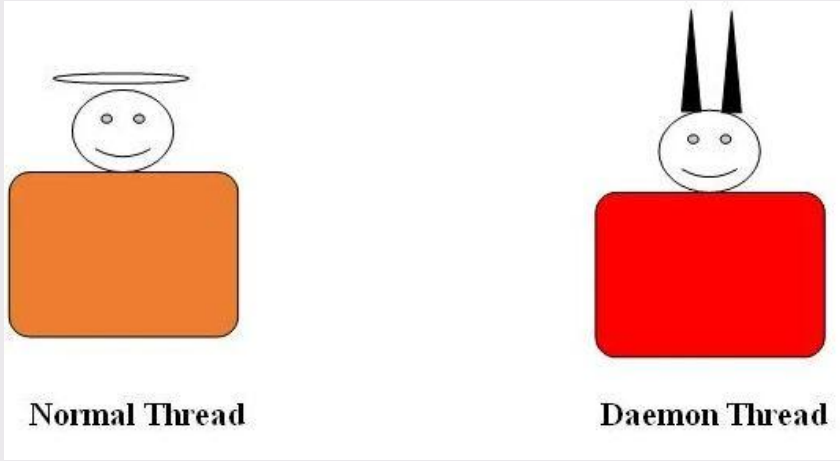
Time Quantum (or) Time Sliced : 04

- While executing the process when that time quantum expires the thread is made to wait state and all threads that are equal priority get their chances to use their quantum in round robin fashion.

Daemon thread

Daemon thread is a low-priority thread which is executing in the background that performs background operations such as garbage collection, finalizer, Action Listeners etc.

The main objective of daemon thread is to provide support for non-daemon threads (main thread)



Eg: If main thread runs with low memory then JVM runs garbage collector to destroy useless objects to free up the memory. So that main thread can continue its execution.

Usually, Daemon thread runs with low priority. But based on our requirement we can change the priority of daemon thread to high priority.

Methods in Daemon Thread

boolean isDaemon(): This method is used to check whether the current thread is a daemon or not. It returns true if the thread is daemon otherwise it returns false.

void setDaemon(boolean status): This method specifies whether the thread should be set as a daemon or a user thread. If true, the thread is set as a daemon thread; if false, the thread is designated as a user thread.

Exceptions in Daemon Thread

Sno	Exception	Description
1	IllegalThreadStateException	If you call the setDeamon() method when the thread is in the running state, it will throw an exception. It should be done before starting of a thread.
2	SecurityException	If the current thread is not able to change this thread

```
public class TestDaemonThread1 extends Thread
{
        public void run()
        {
            if(Thread.currentThread().isDaemon())
            {               //checking for daemon thread
                System.out.println("daemon thread work");
            }
             else
            {
                System.out.println("user thread work");
             }
        }
```

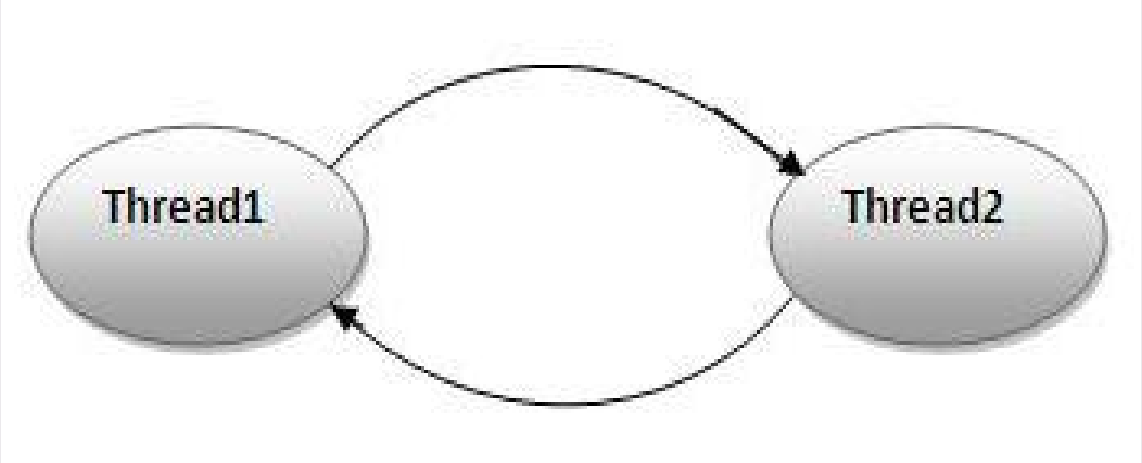
```
public static void main(String[] args)
        {
        TestDaemonThread1 t1=new TestDaemonThread1(); //creating thread
        TestDaemonThread1 t2=new TestDaemonThread1();
        TestDaemonThread1 t3=new TestDaemonThread1();
        t1.setDaemon(true);   //now t1 is daemon thread
        t1.start();   //starting threads
        t2.start();
        t3.start();
        }
}
```

Output:

```
daemon thread work
user thread work
user thread work
```

Deadlock in java

- Deadlock in java is a part of multithreading.
- Deadlock can occur in a situation when a thread is waiting for an object lock, that is acquired by another thread and second thread is waiting for an object lock that is acquired by first thread.
- Since, both threads are waiting for each other to release the lock, the condition is called deadlock.



```
public class TestDeadlockExample1
{
    public static void main(String[] args)
   {
          final String resource1 = "ratan jaiswal";
         final String resource2 = "vimal jaiswal";
         // t1 tries to lock resource1 then resource2
         Thread t1 = new Thread()
         {
             public void run()
             {
                synchronized (resource1)
                {
                    System.out.println("Thread 1: locked resource 1");
                    try
                    {
```

```
                Thread.sleep(100);
             }
            catch (Exception e)
            {
            }
            synchronized (resource2)
            {
                System.out.println("Thread 1: locked resource 2");
            }
        }
    }
};
// t2 tries to lock resource2 then resource1
Thread t2 = new Thread()
      {
            public void run()
            {
```

```
            synchronized (resource2)
            {
            System.out.println("Thread 2: locked resource 2");
                try
                {
                    Thread.sleep(100);
                }
                catch (Exception e)
                {
                }
            synchronized (resource1)
                {
                    System.out.println("Thread 2: locked resource 1");
                }
            }
        }
    };
    t1.start();
    t2.start();
    }
}
```

Output:

```
Thread 1: locked resource 1
Thread 2: locked resource 2
```

ThreadGroup in Java

- Java provides a convenient way to group multiple threads in a single object. In such way, we can suspend, resume or interrupt group of threads by a single method call.
- Java thread group is implemented by *java.lang.ThreadGroup* class.
- There are only two constructors of ThreadGroup class.

Constructor	Description
ThreadGroup(String name)	creates a thread group with given name.
ThreadGroup(ThreadGroup parent, String name)	creates a thread group with given parent group and name.

```
public class ThreadGroupDemo implements Runnable
{
    public void run()
    {
            System.out.println(Thread.currentThread().getName());
    }
    public static void main(String[] args)
    {
        ThreadGroupDemo runnable = new ThreadGroupDemo();
         ThreadGroup tg1 = new ThreadGroup("Parent ThreadGroup");
         Thread t1 = new Thread(tg1, runnable,"one");
         t1.start();
         Thread t2 = new Thread(tg1, runnable,"two");
         t2.start();
         Thread t3 = new Thread(tg1, runnable,"three");
         t3.start();
         System.out.println("Thread Group Name: "+tg1.getName());
         tg1.list();
     }
}
```

Output:

```
one
two
three
Thread Group Name: Parent ThreadGroup
java.lang.ThreadGroup[name=Parent
ThreadGroup,maxpri=10]
Thread[one,5,Parent ThreadGroup]
Thread[two,5,Parent ThreadGroup]
Thread[three,5,Parent ThreadGroup]
```

Thread Synchronization

- Synchronization in java is the capability *to control the access of multiple threads to any shared resource.*
- Java Synchronization is better option where we want to allow only one thread to access the shared resource.

Concept of Lock in Java

- Synchronization is based on the entity known as the lock or monitor. Every object has a lock associated with it.
- A thread that needs to access an object (resource) has to acquire the object's lock before accessing them. It releases the lock when complete it's work with the acquired resource.

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

- There are two types of thread synchronization mutual exclusive and inter-thread communication.
  - Mutual Exclusive
    - Synchronized method.
    - Synchronized block.
  - Cooperation (Inter-thread communication in java)

synchronized block

- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- Scope of synchronized block is smaller than the method.

Syntax for synchronized block

```
synchronized (object )
{
//code block
}
```

```
class Table
{
    void printTable(int n) //method not synchronized
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e){System.out.println(e);}
        }
    }
}
```

```
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
    class MyThread2 extends Thread
    {
        Table t;
        MyThread2(Table t){
        this.t=t;
    }
    public void run()
    {
    t.printTable(100);
    }
}
```

```
class TestSynchronization1
{
    public static void main(String args[])
    {
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Output:

```
5
100
10
200
15
300
20
400
25
500
```

Java synchronized method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

```
class Table
{
    synchronized void printTable(int n)
    {
        //synchronized method
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
}
```

```
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(100);
    }
}
```

```
public class TestSynchronization2
{
    public static void main(String args[])
    {
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Output:

```
5
10
15
20
25
100
200
300
400
500
```

Inter-thread communication in Java

- Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.
- Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed.

It is implemented by following methods of Object class:

- ✓ wait()
- ✓ notify()
- ✓ notifyAll()

wait( ):

This method tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls notify( )

Method	Description
public final void wait() throws InterruptedException	waits until object is notified.
public final void wait(long timeout) throws InterruptedException	waits for the specified amount of time

notify( ):

This method wakes up the first thread that called wait( ) on the same object.

public final void notify()

notifyAll( ):

This method wakes up all the threads that called wait() on the same object. The highest priority thread will run first.

public final void notifyAll()